

inframon

교량 InSAR → PINN 가상센싱 → 위험도 → IFC 디지털 트윈

사용자 워크플로우 안내서

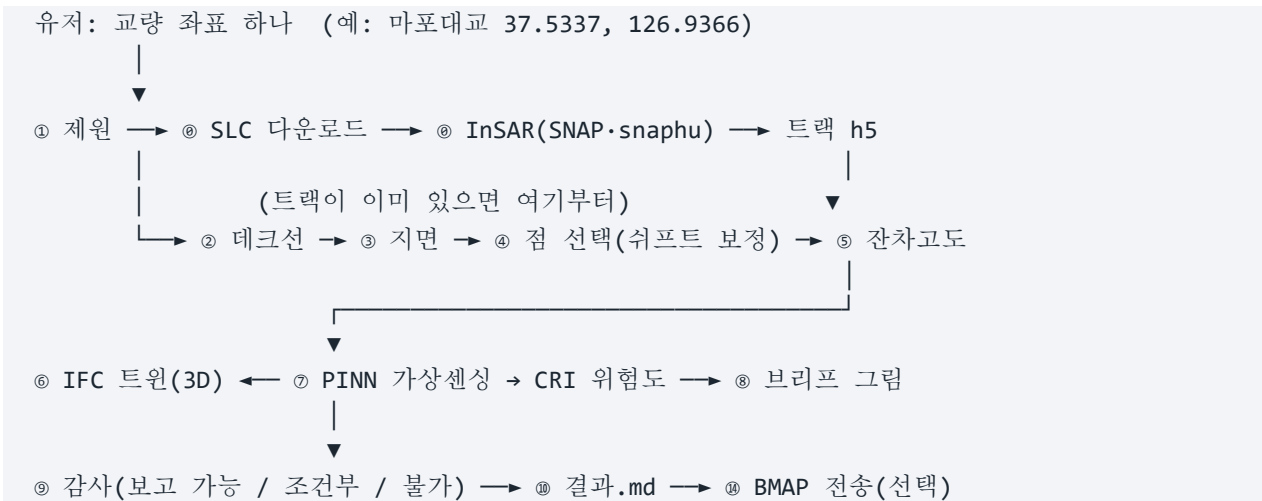
2026-09-09 · <https://github.com/tjddnr8334-sudo/inframon>

1. 이 프로그램이 하는 일

위성 레이더(Sentinel-1 InSAR)로 **교량의 미세 변위**를 재고, 그 관측을 물리 모델(PINN)에 넣어 **관측점이 없는 곳까지 변위장**을 채우고, 그것으로 **위험도(CRI)**를 내고, 전부를 **3D 디지털 트윈(IFC)** 위에 올립니다. 사용자가 하는 것은 **교량을 고르는 것**뿐입니다.

단계	무엇을	어디서
① 교량 선정	좌표 하나 → 연장·경간·폭·형식·형하고	파트너 실측 CSV → OSM → 표준데이터
② SLC 확보	Sentinel-1 SLC 검색·다운로드	ASF (Earthdata 토큰 필요)
③ InSAR	간섭도 → 위상 언래핑 → 변위 시계열	SNAP + snaphu
④ PS/DS	교량 위 산란체 선별, 쉬프트 보정	inframon
⑤ 잔차고도	점이 지면이 아니라 교면 위임 을 고도로 증명	inframon (수직기선 B _⊥)
⑥ IFC 트윈	실측 제원 → IFC4 → 점을 부재에 결합 → 3D	inframon
⑦ PINN-CRI	가상센싱(전체 변위장) → 위험도·경보	inframon
⑧ 브리프	건기연 형식 4 단 그림 (a)(b)(c)(d)	inframon
⑨ 감사	이 결과를 보고에 써도 되는가 — 파일이 스스로 판정	inframon
⑭ BMAP	파트너 플랫폼(Pontifex) 전송	HTTP API

2. 전체 흐름



각 단계는 **없는 것은 없다고 적고 넘어갑니다** — 멈추지 않습니다. 어느 교량을 보고에 쓸지는 감사 판정과 결과.md 를 보고 **유저가 정합니다**.

3. 다른 컴퓨터에서 처음부터 (PowerShell)

3.1 준비물

	필요	쓰는 곳
Python 3.11+	python.org — 설치 시 'Add python.exe to PATH' 체크	전부
Git	git-scm.com	받기
Earthdata 계정	urs.earthdata.nasa.gov 가입(무료, 3 분). 토큰은 3.2 가 브라우저를 열어 주면 붙여넣기	① SLC 다운로드
SNAP	3.2 가 1.1 GB 내려받아 무인 설치 (직접: step.esa.int)	① InSAR 처리
snaphu(WSL)	3.2 가 WSL 안에 apt 설치. WSL 이 없으면 설치를 걸고 재부팅 안내	① 언래핑
SLC 보관 폴더	3.2 가 드라이브를 묻고 폴더를 만들 (장당 4~8 GB, 교량당 200~400 GB → 큰 드라이브)	① 다운로드가 떨어지는 곳

트랙이 이미 있는 교량(정자교·청양교·내곡교 등)만 돌리면 **Python·Git 만** 있으면 됩니다. 사람이 직접 해야 하는 것은 **Earthdata 가입** 하나 — 나머지는 3.2 한 줄이 받아서 설치합니다.

3.2 받기·설치 — 원하는 폴더에서 (10~20 분)

PowerShell(Win + X → 터미널)을 열고, **먼저 프로그램을 둘 폴더로 이동**합니다. 그 아래에 `inframon` 폴더가 생깁니다. 그다음 한 줄 — Python·Git 이 없으면 깔고, 받고, 파이썬 패키지 전부와 SNAP·snaphu 를 설치하고, Earthdata 토큰은 브라우저를 열어 붙여넣게 하고, SLC 보관 드라이브를 묻고, 데모와 대시보드까지 띄웁니다.

```
cd E:\                                     # ①
프로그램을 둘 곳
irm https://raw.githubusercontent.com/tjddnr8334-sudo/inframon/main/install.ps1 | iex #
② 전부
```

손으로 하려면 (같은 폴더에서):

```
cd E:\
git clone https://github.com/tjddnr8334-sudo/inframon
cd inframon
python start.py --full --tools
```

확인: 판정: ☒ 코어 동작 가능 과 결과: snap ☒, snaphu ☒, earthdata ☒, slc_dir ☒. `--full` 이 가상환경(.venv)을 만들고 torch·pyproj·scipy·matplotlib·ifcopenshell 까지 깔니다. `--tools` 가 SNAP·snaphu·토큰·SLC 폴더를 준비합니다. 토큰 프롬프트에서 그냥 Enter 로 건너뛰었으면 3.5 로.

이후 명령은 ``python`` 이 아니라 ``.venv\Scripts\python`` 으로 부릅니다 — `start.py` 가 시스템 파이썬이 아니라 `.venv` 안에 설치하기 때문입니다. 그냥 ``python -m inframon`` 은 새 컴퓨터에서 'No module named inframon' 이 납니다.

3.3 이 PC 에 뭐가 있나 (1 분)

```
.venv\Scripts\python -m inframon --doctor
```

확인: **[외부 도구·자격]** 네 줄 — Earthdata / SNAP gpt / snaphu / SLC 보관 폴더 각각 ☒ ☒. ☒ 면 그 줄에 설치법이 적혀 있고, `python start.py --tools` 를 다시 돌리면 빠진 것만 채웁니다.

3.4 먼저 되는 것으로 한 번 (1 분)

```
.venv\Scripts\python scripts\demo_4pm.py
```

정자교를 44 초에 끝까지 돌리고 결과 창 4 개(3D 속도 · 3D 위험도 · 브리프 그림 · 결과.md)를 엽니다.

확인: 터미널에 ⑩ 결과 문서 까지 찍히고 `rc=0`.

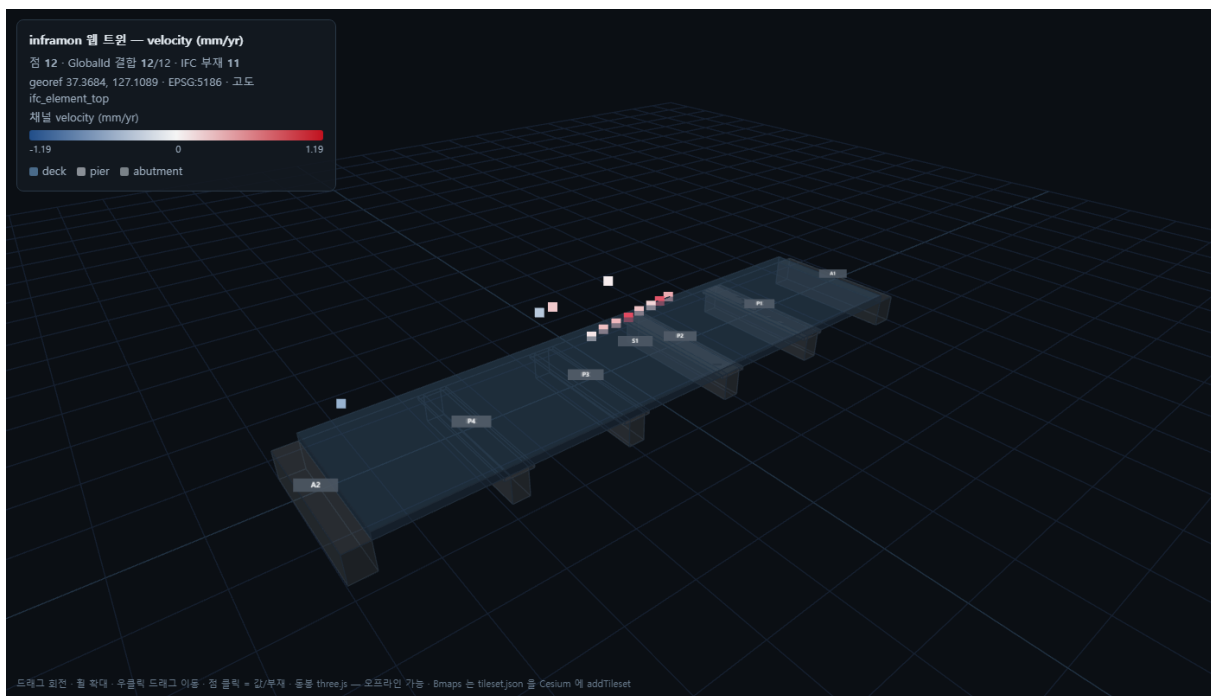


그림 1. 3D 디지털 트윈 — IFC 부재(A1-P1~P4-S1) 위에 InSAR 점. 점을 클릭하면 값·부재·GlobalId.

3.5 토큰·SLC 폴더 — 3.2 에서 건너뛰었다면 (한 번)

```
python start.py --tools --no-demo
```

드라이브 고르기

빠진 것만: 토큰 붙여넣기 · SLC

```
.venv\Scripts\python -m inframon --earthdata-save <토큰>
```

토큰만 따로

```
.venv\Scripts\python -m inframon --slc-dir E:\SLC
```

SLC 폴더만 따로 (없으면 만듦)

토큰은 `~/inframon/earthdata\_token` 에, SLC 폴더는 `~/inframon/config.json` 에 저장됩니다. 확인: `~
-doctor` 에서 Earthdata ☒, SLC 보관 폴더 ☒.

3.6 새 교량 — 계획만 먼저 (1 분, 토큰 불필요)

```
.venv\Scripts\python -m inframon --pipeline 37.5337,126.9366 --pipeline-mode plan --out  
docs\bridges\마포대교\plan
```

확인: ②④ SLC·트랙·프레임 ASC path127 frame120 · 41 장 — 어떤 궤도로 몇 장이 있는지. 장면 수가
나오면 다음으로.

3.7 새 교량 — 끝까지 (1~3 시간)

```
.venv\Scripts\python scripts\bridge_run.py --name 마포대교 --lat 37.5337 --lon 126.9366
```

트랙이 없으므로 ⑩ SLC 다운로드 → SNAP → 언래핑부터 갑니다. 단계마다 찍힙니다:

⑩ SLC → InSAR ← 대부분의 시간 (다운로드 GB 단위 · SNAP 수십 분)
① 제원 ② 테크선 ③ 지면 ④ 점 선택 ⑤ 잔차고도
⑥ IFC 트윈 ⑦ PINN·CRI ⑧ 브리프 ⑨ 감사 ⑩ 결과 문서

확인: `docs\bridges\마포대교\결과.md` — 각 단계가 무엇을 어디서 가져왔는지, 못 한 것은 왜인지.

3.8 결과 보기

파일	무엇
twin.viewer.html	3D 트윈(변위 속도 채널) — 더블클릭, 인터넷 불필요
twin_cri.viewer.html	3D 트윈(CRI 위험도 채널)
brief.png	건기연 형식 4 단 그림 (a)(b)(c)(d)
결과.md	수치 표 · 출처 · 감사 판정 · 적어 둘 것 · 원리상 못 하는 것
*_proxy.ifc	IFC4 프록시 교량 — Revit·BlenderBIM 에서 열림

4. 결과를 어떻게 읽나

4.1 브리프 그림 (a)(b)(c)(d)

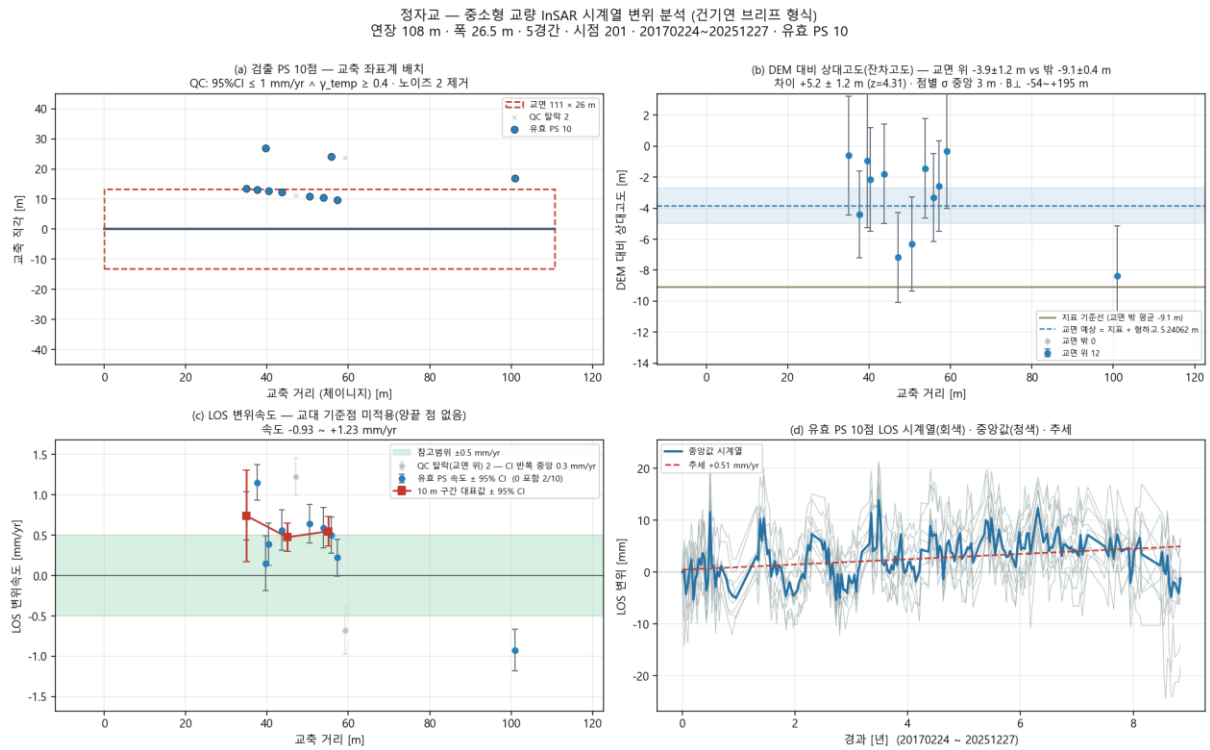


그림 2. 정자교 — (a) PS 교축 배치 (b) 잔차고도: 교면 위 점이 지면보다 $+5.2 \pm 1.2$ m (c) 속도 $\pm 95\%$ CI (d) 시계열·추세

패널	보는 것	판단
(a)	PS 가 교면 안에 몇 개, 어디에	교면 밖 점은 지반 — 교량 값에 섞이면 안 됨
(b)	DEM 대비 상대고도	교면 위 점이 지면보다 형하고만큼 높으면 교면 위 산란체 확인
(c)	속도 $\pm 95\%$ CI, ± 0.5 mm/yr 참고범위	오차막대가 0 을 포함 → 유의한 변형 없음
(d)	시계열 중앙값 · 추세	계절 주기 정상 · 추세 mm/yr

4.2 감사 판정

판정	뜻	예
☒ 보고 가능	연래핑·교량 포함·경간·PINN 물리값 전부 통과	청양교 · 내곡교
☐ 조건부	쓸 수는 있으나 사유가 있음 — 사유를 함께 적을 것	정자교(붕괴 교량, 관측으로 못 봄)
☒ 보고 불가	래핑 위상 · 교량 위 점 0 · 비물리 값	동수원(OSM 연장 불일치)

① 표기는 감점이 아니라 반드시 알아야 할 사실입니다 — 예: '타·고유진동수는 관측값이 아니라 설계 제원 기반'.

4.3 PINN 가상센싱

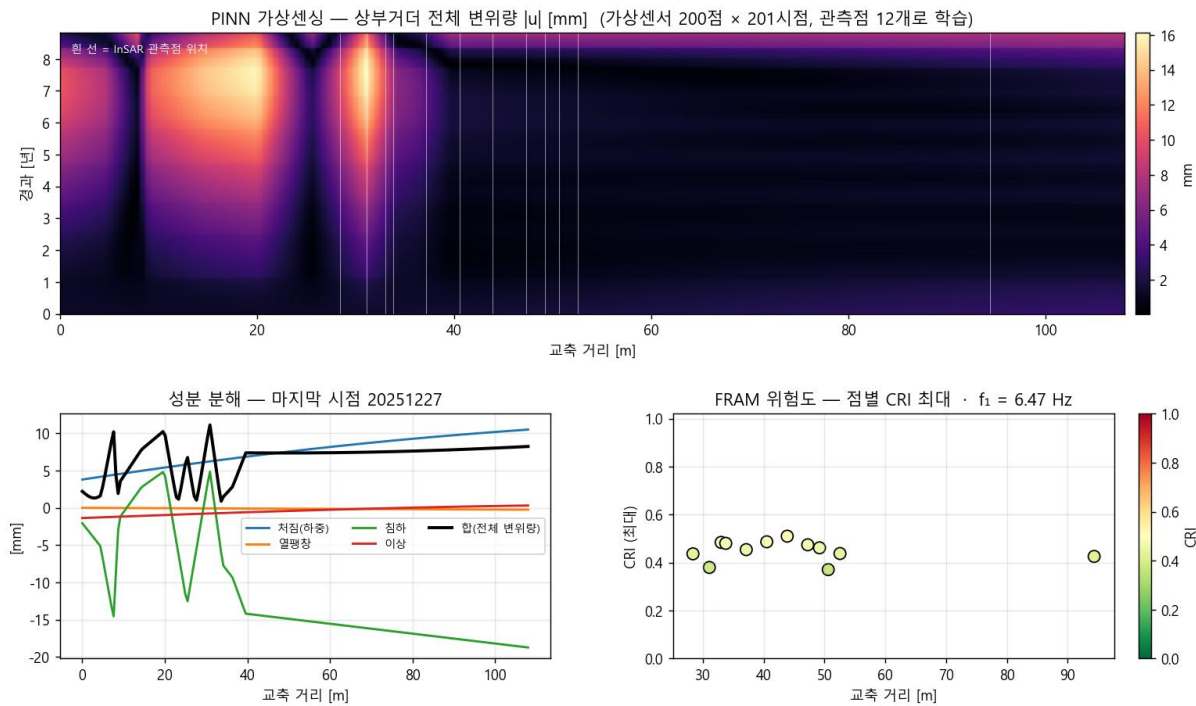


그림 3. 관측점 12 개로 학습한 PINN 이 교축 200 점 × 201 시점의 전체 변위장을 채운다. 흰 선 = 관측점 위치. 관측점 밖은 외삽.

5. 이 프로그램이 원리상 못 하는 것 — 결과를 읽기 전에

항목	왜	그러면
EI(강성) 관측 식별	InSAR 는 상대 변위 — 자중 처짐(청양교 이론 345 mm)은 위성이 보기 전에 이미 들어가 있음	타·고유진동수는 설계 제원 기반 (① 표기). 절대 강성은 레벨링·GNSS 필요
데크 위 PS 밀도	Sentinel-1 화소 ~11 m. 81 m 교량은 교축 구간당 화소 1.3 개	고해상도 SAR·코너리플렉터. 긴 교량(마포대교 1,390 m)은 유리
쉬프트 방향	궤도 heading 이 정함 — 조정 대상 아님	크기 δh 는 잔차고도로 관측값 대체
국부 붕괴 탐지	정자교 2023-04-05 보도부 붕괴(수 m)는 화소보다 작음. 열·추세 제거 후 계단 $z=-0.6$	알려진 사고 교량은 자동 표기하고 '정상'을 보고 근거로 쓰지 않음
시점 수	속도 95% CI 는 장 수가 정함. 25 장 → 2.5 mm/yr, 201 장 → 0.2 mm/yr	브리프 기준 ≥ 100 장. 부족하면 판정 보류

6. 막히면

- ① '토큰 없음' → 3.5 로. 발급은 `urs.earthdata.nasa.gov` (1 분).
- 'SNAP gpt 없음' → SNAP 설치 후 PATH 또는 환경변수 SNAP_HOME.
- 'snaphu 없음' → WSL 에 snaphu. 언래핑이 실패하면 파라미터 사다리 4 단계가 자동으로 돌고, 그래도 안 되면 `unwrap_retry.json` 에 사유.
- OSM 504 → 자동 재시도 3 회. 캐시(`osm_roads_500m.json`)가 있으면 캐시.
- 중간에 죽음 → 같은 명령 다시. 받은 SLC 는 재사용. 결과.md 에 '예외:' 줄이 남음.
- '교면 위 점 0' → 교량이 작거나 궤도 방향이 불리함. 계획(3.6)에서 장면 수·궤도를 먼저 볼 것.

7. 지금까지 돌린 교량 (docs/bridges/)

교량	점	CRI	잔차고도(교면-지면)	감사	비고
청양교	44	0.784	+12.5 ± 6.4 m	보고 가능	25 시점 — 속도 CI 넓음
정자교	12	0.569	+5.2 ± 1.2 m (z 4.3)	조건부	2023-04-05 붕괴 교량 — 관측으로 못 봄
내곡교	34	0.821 경고	—	보고 가능	건기연 브리프 같은 교량 (PS 35 vs 32)
칠백로	22	0.808 경고	—	조건부	
상규	3	0.879 위험	—	보고 가능	점 3 개 — 판정 신뢰도 낮음
동수원	77	0.729	—	보고 불가	CSV 890 m vs OSM 1227 m

여러 교량을 한 번에: ``.\venv\Scripts\python scripts\bridge_run.py --batch docs\bridges\batch.json``